

Pac-Man 3D — Dokumentacja techniczna

1. Cel projektu

Gra przeglądarkowa 3D w stylu Pac-Mana. Gracz porusza się po labiryncie, zbiera kropki, unika duszków i może użyć teleportu (klawisz Spacja), aby przenieść się w losowe wolne miejsce na mapie.

2. Stos technologiczny

Element	Technologia	Rola
Silnik 3D	Three.js r128 (three.min.js)	Scena, kamera, światła, meshe, render WebGL
Język	JavaScript (ES6+)	Cała logika gry w jednym pliku
UI	HTML + CSS	HUD, ekran startowy, przyciski
Grafika	WebGL	Renderowanie w canvas
Serwer (opc.)	Python http.server / URUCHOM.bat	Lokalne serwowanie plików

Gra nie wymaga backendu — działa w całości po stronie klienta (przeglądarki).

3. Struktura plików

```
pacman-3d/  
├── index.html  
├── three.min.js  
├── URUCHOM.bat  
├── dokumentacja-techniczna.pdf  
├── instrukcja-uzytkownika.pdf  
└── README.md
```

4. Architektura kodu (index.html)

Kod JavaScript podzielony jest na sekcje (oznaczone komentarzami):

1. Ustawienia / stałe — CELL, prędkości, życia
2. Mapa labiryntu — tablica stringów MAZE
3. Konwersja współrzędnych — cellToWorld, worldToCell, isWall
4. Scena 3D — renderer, kamera, światła, mgła
5. Podłoga i ściany — InstancedMesh dla wydajności
6. Kropki — obiekty do zbierania
7. Gracz — mesh Pac-Mana + playerPos, facing
8. Duszki — modele 3D + AI po siatce
9. Teleport — losowy wybór komórki z walkableCells
10. Sterowanie — keydown / keyup
11. Kolizje — collidesAt (gracz vs ściany)
12. AI duszków — ghostChooseTarget, updateGhost

13. Stan gry / HUD — wynik, życia, start/koniec

14. Pętla główna — `update(dt)` + `requestAnimationFrame`

5. Reprezentacja mapy

Labirynt to tablica tekstowa MAZE. Każdy znak to jedna komórka:

Znak	Znaczenie
#	Ściana (blokuje ruch)
.	Ścieżka z kropką do zebrania
(spacja)	Puste pole (można chodzić)
P	Start gracza
G	Start duszka

Wymiary: ROWS × COLS. Pozycja w świecie 3D liczona jest funkcją `cellToWorld(r, c)` — środek komórki w układzie współrzędnych Three.js (oś Y w górę).

6. Teleport — implementacja

Przygotowanie listy pól

Przy starcie gry budowana jest tablica `walkableCells` — wszystkie komórki, dla których `isWall()` zwraca `false` (czyli wszystko oprócz # i obszaru poza mapą).

Wywołanie

Klawisz Spacja w `keydown` wywołuje funkcję `teleport()`:

1. Sprawdza, czy gra trwa (`running === true`).
2. Losuje komórkę z `walkableCells` (do 25 prób, żeby uniknąć tego samego miejsca).
3. Przelicza komórkę na współrzędne świata (`cellToWorld`).
4. Ustawia `playerPos` i pozycję mesha gracza.
5. Aktualizuje HUD (TELEPORT: UŻYTO! przez 0,6 s).

Teleport jest natychmiastowy — nie ma animacji lotu ani cooldownu.

7. Ruch gracza

- Wejście: tablica `keys` (które klawisze są wciśnięte).
- Wektor ruchu normalizowany (WASD / strzałki).
- Ruch rozdzielony na osie X i Z — osobno sprawdzana kolizja (`collidesAt`), co daje efekt ślizgania wzdłuż ścian.
- `facing` zapamiętuje ostatni kierunek (obróć model gracza).

8. AI duszków

Duszki poruszają się po siatce (komórka po komórce), nie po ciągłych współrzędnych jak gracz.

Algorytm `ghostChooseTarget`:

1. Z aktualnej komórki wybiera sąsiednie pola (góra/dół/lewo/prawo), pomijając ściany.

2. Unika natychmiastowego zawracania (chyba że ślepy zaułek).
3. W 70% przypadków wybiera komórkę najbliższej gracza.
4. W 30% — losowy kierunek (żeby duszki nie blokowały się nawzajem).

Ruch: interpolacja pozycji mesha w stronę target z prędkością GHOST_SPEED.

9. Kolizje i zasady gry

Zdarzenie	Efekt
Gracz dotyka kropki	+10 pkt, kropka znika
Gracz dotyka duszka	-1 życie, reset pozycji
Wszystkie kropki zebrane	Wygrana
Życia = 0	Przegrana

Kolizje liczone są odległością euklidesową w płaszczyźnie XZ (kwadrat odległości, bez sqrt — wydajniej).

10. Renderowanie i pętla gry

```
requestAnimationFrame(loop)
  → dt = czas od poprzedniej klatki
  → jeśli running: update(dt)
  → renderer.render(scene, camera)
```

- dt ograniczony do max 0,05 s — stabilność przy lagach.
- Kamera: lerp w stronę pozycji gracza + offset CAM_OFFSET (widok z góry-tyłu).
- Cienie: DirectionalLight + shadowMap na rendererze.

11. Wymagania i uruchomienie

- Przeglądarka z WebGL (Chrome, Firefox, Edge).
- Plik three.min.js w tym samym folderze co index.html.
- Zalecany lokalny serwer HTTP (URUCHOM.bat lub python -m http.server 8765).

12. Możliwe rozszerzenia

- Cooldown teleportu (np. 3 s)
- Dźwięki (Web Audio API)
- Więcej poziomów (osobne tablice MAZE)
- Power-upy (np. chwilowa nietykalność po teleporcie)
- Zapis najlepszego wyniku (localStorage)